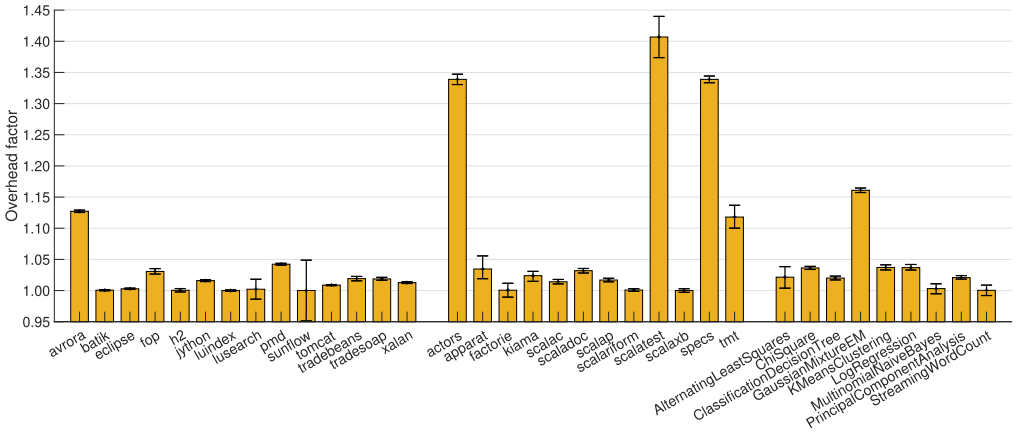


Fig. 7. Profiling overhead on DaCapo (first group), ScalaBench (second group), and Spark Perf (third group).

8.3 Perturbations

The instrumentation code inserted for profiling the metrics of interest causes the execution of additional computations that are not part of the original observed application. While this approach is necessary to collect the desired metrics, the computations inserted may alter the metrics collected, leading to results that may not be representative of the original application behavior. Among all metrics profiled by tgp, reference cycles (used as a measure of task granularity) are particularly susceptible to perturbations caused by the profiling logic, because the additional computations inserted by the instrumentation may result in extra cycles elapsed during the execution of a task, which may be accounted in the granularity of the task. Reducing the amount of extra cycles elapsed due to instrumentation code is therefore very important to collect accurate task-granularity profiles.

The goal of this section is to estimate the amount of extra cycles introduced in the observed application when profiling task granularity with tgp. To this end, we measure the total amount of reference cycles elapsed when executing the observed application with and without tgp enabled. The measured reference cycles represent both the sequential and parallel work performed on all cores during the execution of an application. We collect reference cycles by attaching perf to the observed JVM, using a configuration that enables profiling the amount of cycles elapsed during program execution, regardless of whether tgp is used. We do not consider cycles elapsed during garbage collection, when no application- and framework-layer tasks are in execution.

To relate the amount of extra cycles introduced by tgp to the cycles originally elapsed in the target application, we introduce a new metric, the *perturbation factor*, defined as the number of reference cycles elapsed with tgp enabled, divided by the number of cycles elapsed with tgp disabled. We use perturbation factors as an indication of the quality of the collected profiles. High perturbation factors indicate that the collected metrics are likely to be biased, while low perturbation factors increase our confidence in the results. Note that the purpose of this metric is not to exactly quantify the (relative) amount of extra cycles inserted by tgp in the collected task-granularity profiles. A high perturbation factor indicates that a significant amount of extra cycles has been introduced by the profiling logic; however, such cycles are not necessarily all counted in the collected task profiles. On the other hand, a low perturbation factor does not guarantee that the metrics are perturbed only little, as tgp might overestimate the granularity of some tasks while underestimating the granularity of some other tasks, without this being reflected in a high